Assignment 9 - Git Branching

TW Lab 2025

Name: Saarvik Singh Suryavanshi
ID: 2025UCP1681
Batch: A3

Step 1: View the current commit history (git log --oneline)

```
PS D:\Assignment 9> git log --oneline
f8b9f75 (HEAD -> main, origin/main) Assignment 9 initialization
5ee4eff Removed readme
2b0354f Added vecadd.py
8ec6f99 Revert "After modification"
2507258 After modification
712040f Adding first file.
```

I ran git log --oneline to view the repository history. The output lists the commits with their short hashes and messages, with the latest commit shown first. HEAD points to the current main branch.

This output shows the earlier commits in the repository, including the initial file and later README changes. It provides the commit history used before creating the new branch.

Step 2: Create and verify the fast branch (git branch)

```
PS D:\Assignment 9> git branch fast
PS D:\Assignment 9> git branch
  fast
* main
```

I created a new branch named fast using git branch fast. Running git branch listed the available branches; the asterisk indicates that main was still the active branch.

Step 3–4: Rename the branch and switch to parallel

```
PS D:\Assignment 9> git branch -m fast parallel
PS D:\Assignment 9> git checkout parallel
Switched to branch 'parallel'
PS D:\Assignment 9> git status
On branch parallel
nothing to commit, working tree clean
```

I renamed fast to parallel using git branch -m fast parallel. I then switched to parallel with git checkout parallel. The git status output confirms that parallel is the active branch and the working tree is clean.

Step 5: Commit the parallel implementation and compare branch history

```
PS D:\Assignment 9> git log
commit 86749b4ea54c6c26f8124acdee9714c8ec06c9ae (HEAD -> parallel, origin/parallel)
Author: Saarvik-got-it <saarvikxcode@gmail.com>
Date:   Thu Nov 12 15:36:24 2026 +0530

    Added parallelism

commit f8b9f754d1e61989fd24135989a475ada61ed634 (origin/main, main)
Author: Saarvik-got-it <saarvikxcode@gmail.com>
Date:   Thu Nov 12 15:25:07 2026 +0530

    Assignment 9 initialization

commit 5ee4effdb055da7e816d8f5698c8a92dcdadfa7b
Author: Saarvik-got-it <saarvikxcode@gmail.com>
Date:   Thu Oct 29 18:29:26 2026 +0530

    Removed readme

commit 2b0354fb952293f5731d2ac3d1db1e6a1c166062
Author: Saarvik-got-it <saarvikxcode@gmail.com>
Date:   Thu Oct 29 18:28:44 2026 +0530

    Added vecadd.py

commit 8ec6f9923a3cbcd0e60d1fc0e65a3cd455ee1567
Author: Saarvik-got-it <saarvikxcode@gmail.com>
Date:   Thu Oct 29 18:26:04 2026 +0530

    Revert "After modification"

    This reverts commit 25072584256dba51bcd84ab30ab17302c797cca9.
```

```
commit 25072584256dba51bcd84ab30ab17302c797cca9
Author: Saarvik-got-it <saarvikxcode@gmail.com>
Date:   Thu Oct 29 18:25:26 2026 +0530

    After modification

commit 712040fa5bf86e321b3cec0664530583d72b6cee
Author: Saarvik-got-it <saarvikxcode@gmail.com>
Date:   Thu Oct 29 18:16:36 2026 +0530

    Adding first file.
```

The log shows the new parallelism commit on the parallel branch, while main retains its earlier history. The branch labels show that the two branches have different latest commits at this point.

Step 6: Commit the bug fix on the main branch

```
PS D:\Assignment 9> git add .
PS D:\Assignment 9> git commit -m "Bug Fix in friend branch"
[main 192ea71] Bug Fix in friend branch
 1 file changed, 1 insertion(+), 1 deletion(-)
```

I switched to the main branch, staged the code change with git add ., and committed it with the message “Bug Fix in friend branch”. Git confirms that the commit was created successfully.

The output confirms the bug-fix commit on main and displays its new commit hash.

```
PS D:\Assignment 9> git log --oneline --all --graph
* 192ea71 (HEAD -> main) Bug Fix in friend branch
| * 86749b4 (origin/parallel, parallel) Added parallelism
|/
* f8b9f75 (origin/main) Assignment 9 initialization
* 5ee4eff Removed readme
* 2b0354f Added vecadd.py
* 8ec6f99 Revert "After modification"
* 2507258 After modification
* 712040f Adding first file.
PS D:\Assignment 9> git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
PS D:\Assignment 9>
```

The graph displays the separate commit histories of main and parallel. The following git status output confirms that main is active and the working directory is clean.

Step 7–8: Merge parallel and handle the merge conflict

```
PS D:\Assignment 9> git merge parallel
Auto-merging vecadd.py
CONFLICT (content): Merge conflict in vecadd.py
Automatic merge failed; fix conflicts and then commit the result.
```

I attempted to merge parallel into the current branch. Git reported a **conflict** in vecadd.py, so the automatic merge could not be completed. I manually checked and decided which code to keep and recommitted the file successfully.

```
PS D:\Assignment 9> git commit -m "Bug Fix in friend branch"
[main 59384b6] Bug Fix in friend branch
```

Step 9: Delete the parallel branch

```
PS D:\Assignment 9> git branch -d parallel
Deleted branch parallel (was 86749b4).
```

After merging the changes, I deleted the parallel branch using **git branch -d parallel**. Git confirmed that the branch was deleted.

Step 10: Explore Git stash

The assignment asks us to explore **git stash**, which temporarily saves uncommitted changes so that the working directory can be used for another branch or commit. The command **git help stash** opens the help page describing its usage.

```
PS D:\Assignment 9> git stash
No local changes to save
```